SERVICIOS WEB DISTRIBUIDOS

SISTEMAS DISTRIBUIDOS

Carlos Rojas Sánchez

Licenciatura en Informática

Universidad del Mar

Contenido

1. Introducción

2. Arquitectura Orientada a Servicios (SOA)

3. Definición de Servicio Web

4. Arquitectura de un Servicio Web

5. Tecnologías de Servicios Web

6. Computación en la Nube (Cloud Computing)

7. Características de la Nube

8. Modelos de Servicio

Introducción

Introducción a los Servicios Web

- Evolución de los sistemas distribuidos hacia la web.
- **Definición:** Aplicaciones modulares accesibles vía red.
- Utilizan estándares abiertos y protocolos de Internet.
- Independencia total de plataforma y lenguaje.
- Basados en el modelo Cliente-Servidor sobre HTTP.
- Facilitan la integración de sistemas heterogéneos.
- Piedra angular de la arquitectura orientada a servicios (SOA).

¿Por qué Servicios Web?

- **Interoperabilidad:** Conectar Java, .NET, Python y PHP.
- **Ubicuidad:** HTTP es permitido por casi todos los firewalls.
- **Bajo Acoplamiento:** El cliente no necesita la clase del servidor.
- **Reutilización:** Servicios expuestos para múltiples clientes.
- **Escalabilidad:** Facilidad para desplegar en la nube.
- Sustituyen la complejidad de CORBA por la simplicidad de la Web.

Componentes de la Arquitectura

- **Service Provider:** Expone la interfaz del servicio.
- **Service Requestor:** Consume el servicio (Cliente).
- **Service Description:** Contrato que define cómo interactuar.
- **Transporte:** Generalmente HTTP/HTTPS.
- **Mensajería:** Formato de los datos (XML o JSON).
- **Descubrimiento:** Mecanismos para localizar el servicio.

Tipos de Servicios Web

- **Big Web Services (SOAP):**
- Basados en verbos y protocolos rígidos (XML).
- Enfoque en seguridad y transacciones complejas.
- **RESTful Web Services:**
- Basados en recursos y sustantivos (URI).
- Ligeros, utilizan los métodos nativos de HTTP.
- **Microservicios:**
- Evolución hacia servicios granulares y autónomos.

Arquitectura Orientada a Servicios (SOA)

Concepto de Arquitectura Orientada a Servicios

- Estilo de diseño de software basado en servicios discretos.
- Los servicios se comunican a través de protocolos estándar.
- **Servicio:** Unidad lógica que resuelve un problema de negocio.
- No es una tecnología, es un **modelo arquitectónico**.
- Fomenta el alineamiento entre TI y objetivos de negocio.
- Evolución natural de la Computación Distribuida.
- Base para la agilidad y flexibilidad organizacional.

Principios Fundamentales de SOA

- **Contrato de Servicio:** Descripción formal del servicio.
- **Bajo Acoplamiento:** Independencia entre consumidor y proveedor.
- **Abstracción:** La lógica interna está oculta al exterior.
- **Reutilización:** Diseñados para usarse en múltiples contextos.
- **Autonomía:** El servicio controla sus propios recursos.
- **Sin Estado (Statelessness):** Minimiza la retención de datos.
- **Descubribilidad:** Deben ser localizables mediante metadatos.

El Modelo de Interacción SOA

- **Proveedor (Provider):** Crea y publica el servicio.
- **Consumidor (Consumer):** Busca y utiliza el servicio.
- **Directorio (Registry):** Repositorio donde se anuncian.
- **Operación Find:** El cliente busca en el directorio.
- **Operación Bind:** El cliente se conecta al proveedor.
- **Operación Publish:** El proveedor se registra en el directorio.

Ventajas de Implementar SOA

- **Interoperabilidad:** Une plataformas heterogéneas.
- **Mantenibilidad:** Cambios internos no afectan al cliente.
- **Escalabilidad:** Se pueden replicar servicios específicos.
- **Reducción de Costos:** Menor tiempo de desarrollo (reúso).
- **Flexibilidad:** Rápida respuesta a cambios en el mercado.

Mensajería en SOA: El pegamento del sistema

- La comunicación se basa en el intercambio de **Mensajes**.
- El mensaje es una unidad de datos independiente.
- Debe contener metadatos (cabeceras) y carga útil (body).
- **Formatos comunes:** XML (SOAP) y JSON (REST).
- El transporte suele ser HTTP, pero puede ser MQ o SMTP.
- Soporta patrones síncronos y asíncronos.

Tipos de Mensajería: Síncrona vs Asíncrona

- **Síncrona (Request-Response):**
- El cliente bloquea la ejecución hasta recibir respuesta.
- Ejemplo: Consulta de saldo bancario.
- **Asíncrona (Fire-and-Forget / Callback):**
- El cliente envía el mensaje y continúa su trabajo.
- Ideal para procesos largos o desacoplados.
- Ejemplo: Procesamiento de una orden de compra compleja.

El Enterprise Service Bus (ESB)

- Componente de software que actúa como intermediario.
- **Funciones clave:**
- Enrutamiento de mensajes.
- Transformación de datos (ej. XML a JSON).
- Manejo de protocolos (ej. FTP a HTTP).
- Seguridad y autenticación centralizada.
- Centraliza la lógica de integración, no de negocio.

Composición de Servicios (Orquestación)

- SOA permite crear servicios complejos a partir de simples.
- **Orquestación:** Un proceso central dirige a los demás.
- Similar a un director de orquesta.
- **Coreografía:** Cada servicio sabe cuándo actuar.
- No hay un controlador central; es una danza coordinada.
- Crucial para procesos de negocio de múltiples pasos.

Desafíos en la implementación de SOA

- **Complejidad:** Gestión de múltiples interfaces.
- **Rendimiento:** Sobrecarga por serialización (XML/JSON).
- **Gobernanza:** ¿Quién es dueño de qué servicio?
- **Seguridad:** Mayor superficie de ataque distribuida.
- **Pruebas:** Difícil de testear de forma aislada.

Métricas de Calidad en SOA

- **Granularidad:** ¿Qué tanto hace un solo servicio?
- **Acoplamiento:** Nivel de dependencia entre módulos.
- **Cohesión:** Enfoque del servicio en una sola tarea.
- **Disponibilidad:** Tiempo de actividad del servicio.
- **Latencia:** Tiempo de respuesta en la red.

SOA vs. Monolitos

- **Monolito:** Una sola unidad de despliegue y código.
- Difícil de escalar y actualizar parcialmente.
- **SOA:** Múltiples unidades independientes.
- Facilita el despliegue continuo.
- Permite usar diferentes tecnologías en cada servicio.

Preparación del Entorno de Ejecución

- Se requiere Python 3.x y el gestor de paquetes `pip`.
- Instalación de dependencias (Flask para el Servidor, Requests para el Cliente):

```
$ pip install flask requests
```

- **Sugerencia:** Utilizar un entorno virtual (`venv`) para mantener la limpieza del sistema.

Implementación: Definición del Servicio

- Usaremos un enfoque simple basado en **Python y Flask**.
- **Servicio:** Consultar clima por ciudad.
- El contrato es la URL y el método HTTP.

Implementación: El Proveedor (Provider)

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/clima/<ciudad>', methods=['GET'])
def get_clima(ciudad):
    # Logica de negocio abstracta
    datos = {"Oaxaca": "28C", "Cancun": "30C"}
    resultado = datos.get(ciudad, "Desconocido")
    return jsonify({"ciudad": ciudad, "clima": resultado})

if __name__ == '__main__':\
    app.run(port=5000)
```

Implementación: El Consumidor (Consumer)

```
import requests

def consultar_servicio(ciudad):
    url = f"http://localhost:5000/clima/{ciudad}"
    try:
        response = requests.get(url)
        data = response.json()
        print(f"Clima en {data['ciudad']}: {data['clima']}")
    except Exception as e:
        print(f"Error en la comunicacion: {e}")

consultar_servicio("Oaxaca")
```

Paso 1: Lanzar el Proveedor de Servicios

1. Guardar el código del servidor en `provider.py`.
2. Ejecutar desde la terminal:

```
$ python provider.py
 * Serving Flask app 'provider'
 * Debug mode: off
 * Running on http://127.0.0.1:5000
```

- El servicio ahora está "Publicado" y escuchando peticiones en el puerto 5000.

Paso 2: Prueba de Interoperabilidad (CURL)

- Antes de usar el cliente Python, validamos el "Contrato" usando una herramienta agnóstica como `curl`.

```
$ curl http://localhost:5000/clima/Oaxaca
```

- **Respuesta esperada (JSON):**

```
{
  "ciudad": "Oaxaca",
  "clima": "28C"
}
```

Paso 3: Ejecución del Consumidor

1. Abrir una **segunda terminal** (manteniendo el servidor activo).
2. Guardar el código del cliente en `consumer.py` y ejecutar:

```
$ python consumer.py
Clima en Oaxaca: 28C
```

- El cliente ha realizado un "Find" (vía URL) y un "Bind" (vía HTTP) exitosamente.

¿Qué sucedió detrás de escena?

- **Mensajería:** El cliente empaquetó una petición HTTP GET.
- **Transporte:** Los datos viajaron por TCP/IP al puerto 5000.
- **Procesamiento:** Flask recibió el mensaje, extrajo el parámetro "ciudad" y ejecutó la lógica.
- **Respuesta:** El servidor serializó el diccionario Python a un formato **JSON** (estándar SOA).
- **Desacoplamiento:** Si el servidor cambia de Python a Node.js, el cliente no lo notaría mientras la URL sea la misma.

Resumen

- SOA es un cambio de mentalidad: de "objeto" a "servicio".
- La interoperabilidad es su mayor fortaleza.
- Se apoya en mensajería estándar (XML/JSON).
- El bajo acoplamiento permite evolucionar el sistema.
- Es la base conceptual para entender Microservicios.

Definición de Servicio Web

¿Qué es un Servicio Web?

- **Definición según el W3C:**
- Sistema de software diseñado para soportar la interacción máquina-a-máquina interoperable a través de una red.
- Tiene una interfaz descrita en un formato procesable por máquinas.
- Otros sistemas interactúan con el servicio web de la manera prescrita por su descripción.
- Utiliza mensajes basados en estándares (XML, JSON).
- Se transporta típicamente sobre el protocolo HTTP.

Características de un Web Service

- **Autodescriptivo:** Publica su contrato (WSDL, OpenAPI/Swagger).
- **Modular:** Realiza una tarea específica y bien definida.
- **Basado en Texto:** Los mensajes son legibles (no binarios como RMI).
- **Pasivo:** El servicio espera a ser invocado (modelo Request/Response).
- **Agnóstico:** No depende de un lenguaje de programación específico.
- **Abierto:** Utiliza estándares de la industria (IETF, W3C).

La "Pila de Protocolos" del Servicio Web

- **Transporte:** TCP, HTTP, HTTPS, SMTP (Capa de Red).
- **Mensajería:** XML, JSON, YAML (Formato de Datos).
- **Descripción:** WSDL, OpenAPI (El Contrato).
- **Descubrimiento:** UDDI, DNS, Directorios (Localización).
- **Seguridad:** SSL/TLS, OAuth, WS-Security.

Web Tradicional vs. Servicio Web

Característica	**Web Tradicional**	**Servicio Web**
Consumidor	Humano (Navegador)	Máquina (Aplicación)
Contenido	HTML / CSS / JS	XML / JSON
Propósito	Presentación visual	Intercambio de datos
Protocolo	HTTP / HTTPS	HTTP / HTTPS / SOAP

El Contrato del Servicio

- Un servicio web moderno se define por su **Endpoint** y su estructura de datos.

```
Servicio: Consulta de Alumnos (UMAR)
Endpoint: http://api.umar.edu.mx/v1/alumnos/{matricula}
Metodo: GET
Respuesta Esperada:
{
  "matricula": "20230101",
  "nombre": "Juan Perez",
  "carrera": "Lic. Informática"
}
```

Arquitectura de un Servicio Web

Modelo de Referencia de un Web Service

- Estructura lógica que define la interacción entre componentes.
- Basada en estándares abiertos de la W3C y OASIS.
- Propósito: Garantizar la interoperabilidad total.
- Independiente de la implementación interna (Caja Negra).
- Se fundamenta en el intercambio de mensajes estructurados.
- Facilita la integración de sistemas heredados (Legacy).

Roles en la Arquitectura de Web Services

- **Proveedor de Servicio (Provider):** Quien posee el recurso.
- **Consumidor de Servicio (Requester):** Quien solicita el recurso.
- **Agente de Descubrimiento (Broker/Registry):** El intermediario.
- Estos roles forman el "Triángulo de Interacción".
- Un mismo sistema puede actuar como proveedor y consumidor.

Rol: Proveedor de Servicio

- Implementa la lógica de negocio del servicio.
- Define el contrato (Interfaz técnica).
- Publica la descripción en el Directorio/Broker.
- Escucha peticiones y envía respuestas (Endpoint).
- Gestiona la seguridad y el acceso a los datos.

Rol: Consumidor de Servicio

- Localiza el servicio mediante el Broker o una URL directa.
- "Encuaderna" (Bind) su lógica con la interfaz del proveedor.
- Envía mensajes de petición según el formato acordado.
- Procesa la respuesta recibida (Parsing).
- Puede ser una App móvil, otro Servidor o un Script.

Rol: Broker de Servicios (Service Registry)

- Repositorio central de descripciones de servicios.
- Permite la búsqueda por funcionalidad o categoría.
- Almacena metadatos (WSDL, esquemas, políticas).
- Históricamente implementado mediante UDDI.
- En la web moderna, suele ser un portal de APIs o DNS.

Pila de Protocolos de Web Services

- Conjunto de capas necesarias para la comunicación.
- **Capa de Transporte:** Envío físico de datos.
- **Capa de Mensajería:** Estructura de la información.
- **Capa de Descripción:** Definición de la interfaz.
- **Capa de Descubrimiento:** Localización del servicio.

1. Capa de Transporte

- Responsable de mover mensajes entre aplicaciones.
- **Protocolo Rey:** HTTP / HTTPS (Puertos 80 / 443).
- Otros: SMTP, FTP, BEEP, o Colas de Mensajes (JMS).
- HTTP es preferido por su capacidad de atravesar firewalls.
- Provee el método de entrega (GET, POST, etc.).

2. Capa de Mensajería XML / JSON

- Define el formato de los datos intercambiados.
- **XML (SOAP):** Estricto, basado en etiquetas personalizadas.
- **JSON (REST):** Ligero, basado en pares clave-valor.
- Ambos son legibles por humanos y máquinas.
- Garantizan la independencia del lenguaje de programación.

3. Capa de Descripción de Servicio

- Especifica "qué" hace el servicio y "cómo" invocarlo.
- **WSDL (Web Services Description Language):** Para SOAP.
- **OpenAPI / Swagger:** Para servicios REST.
- Incluye: Tipos de datos, operaciones y endpoints.
- Es el plano técnico para los desarrolladores.

4. Capa de Descubrimiento y Otros

- **Descubrimiento:** Mecanismos de búsqueda (UDDI).
- **Seguridad:** Firmas digitales y cifrado (WS-Security).
- **Transacciones:** Coordinación de múltiples servicios.
- **Orquestación:** Composición de flujos de trabajo (BPEL).

Ciclo de Vida del Web Service

- **Definición:** Identificar la necesidad de negocio.
- **Diseño:** Especificar la interfaz (Contrato).
- **Implementación:** Codificar la lógica y el servidor.
- **Despliegue:** Publicar en un servidor accesible.
- **Publicación:** Registrar en el directorio.
- **Mantenimiento:** Versionado y monitoreo.

Enfoque Contract-First (Top-Down)

- Se diseña primero el contrato (WSDL o Swagger).
- Se genera el código base (Scaffolding) a partir del diseño.
- Ventaja: Interfaz limpia y desacoplada del código.
- Recomendado para equipos grandes y estandarización.

Enfoque Code-First (Bottom-Up)

- Se escribe primero el código de la lógica de negocio.
- Las herramientas generan el contrato automáticamente.
- Ventaja: Rapidez en prototipado y facilidad inicial.
- Riesgo: El contrato puede ser frágil ante cambios de código.

Stack Tecnológico para Desarrollo

- **Java:** JAX-WS, Spring Boot.
- **Python:** Flask, FastAPI, Django REST Framework.
- **Servidores:** Apache, Nginx, Gunicorn, Tomcat.
- **Pruebas:** Postman, Insomnia, SoapUI, curl, bruno.
- **Serialización:** Librerías JSON/XML nativas.

Resumen

- La arquitectura es una red de acuerdos técnicos.
- Los roles (Provider/Consumer) definen la responsabilidad.
- La pila de protocolos asegura el viaje del mensaje.
- El éxito del desarrollo depende del diseño del contrato.

Tecnologías de Servicios Web

Evolución de las Tecnologías Web

- Transición de RPC binario a protocolos basados en texto.
- Necesidad de estándares abiertos para la interoperabilidad.
- La pila WS (Web Services Stack) define el "cómo".
- **XML-RPC:** El precursor simplificado.
- **SOAP:** El estándar robusto de mensajería.
- **WSDL:** La gramática y descripción técnica.
- **UDDI:** El catálogo de localización global.

XML-RPC: El Origen

- Creado por Dave Winer en 1998.
- Protocolo de llamada a procedimiento remoto usando XML.
- **Transporte:** HTTP (usualmente POST).
- **Mensajería:** XML sencillo.
- Ventaja: Extremadamente simple de implementar.
- Desventaja: Tipado limitado y falta de estándares de seguridad.

Estructura de una Petición XML-RPC

```
<?xml version="1.0"?>
<methodCall>
  <methodName>matematicas.sumar</methodName>
  <params>
    <param><value><int>10</int></value></param>
    <param><value><int>20</int></value></param>
  </params>
</methodCall>
```

SOAP (Simple Object Access Protocol)

- Protocolo estándar para el intercambio de información.
- Basado estrictamente en XML.
- Diseñado para ser extensible y neutral al transporte.
- Mantenido por la W3C.
- **Filosofía:** Comunicación basada en "Verbos" (Acciones).
- Soporta patrones de comunicación complejos (Sinc/Asinc).

Características de SOAP

- **Independencia:** Funciona sobre HTTP, SMTP, TCP.
- **Neutralidad:** No depende de un lenguaje (Java, .NET, etc.).
- **Extensibilidad:** Mediante el uso de "WS-*" (Seguridad, Routing).
- **Tipado Fuerte:** Basado en esquemas XML (XSD).
- **Manejo de Errores:** Estándar mediante `Fault`.

La Anatomía de un Mensaje SOAP

- **Envelope (Sobre):** Raíz del documento XML.
- **Header (Cabecera):** Datos de control, seguridad, ruteo.
- **Body (Cuerpo):** Los datos reales de la petición o respuesta.
- **Fault:** Información de errores (opcional).

Ejemplo de Mensaje SOAP (Request)

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <auth:Token>ABC-123</auth:Token>
  </soap:Header>
  <soap:Body>
    <m:GetAlumno xmlns:m="http://umar.edu.mx/alumnos">
      <m:Matricula>2023050</m:Matricula>
    </m:GetAlumno>
  </soap:Body>
</soap:Envelope>
```

WSDL (Web Service Description Language)

- Documento XML que describe un Servicio Web.
- Es el "Contrato" entre el proveedor y el consumidor.
- Indica **dónde** está el servicio y **qué** hace.
- Describe los formatos de los mensajes de entrada y salida.
- Permite la generación automática de código (Proxy/Stubs).

Secciones de un Documento WSDL

- **Types:** Definición de tipos de datos (XSD).
- **Message:** Definición de los datos transferidos.
- **PortType:** Operaciones abstractas del servicio.
- **Binding:** Protocolo de comunicación (ej. HTTP-SOAP).
- **Service/Port:** El Endpoint (URL) del servicio.

WSDL: El Plano de la Interfaz

- **Abstracto:** Define el "Qué" (Mensajes y Operaciones).
- **Concreto:** Define el "Cómo" y "Dónde" (Binding y Dirección).
- Sin el WSDL, el cliente no sabe cómo construir el XML de SOAP.

UDDI (Universal Description Discovery and Integration)

- Directorio centralizado para buscar servicios web.
- "Páginas Amarillas" para aplicaciones empresariales.
- Permite a las empresas descubrir servicios de socios.
- Basado en XML y SOAP.
- Aunque ha perdido fuerza ante directorios privados y REST, su concepto persiste en los *Service Registries*.

Tipos de Información en UDDI

- **White Pages:** Datos de la empresa (Nombre, contacto).
- **Yellow Pages:** Categorización (Industria, ubicación).
- **Green Pages:** Información técnica (Referencias a WSDL).
- Permite la interoperabilidad dinámica en B2B.

Flujo de Trabajo: Publish, Find, Bind

1. **Publish:** El proveedor registra el WSDL en UDDI.
2. **Find:** El cliente busca el servicio en UDDI y obtiene el WSDL.
3. **Bind:** El cliente usa el WSDL para generar un mensaje SOAP y llamar al proveedor.

Ventajas de esta Pila Tecnológica

- **Rigidez Formal:** Los errores se detectan en compilación.
- **Seguridad Integrada:** WS-Security para firmas y cifrado.
- **Transaccionalidad:** WS-AtomicTransaction.
- **Legibilidad:** Todo es texto transparente a la red.
- Ideal para sistemas bancarios y gubernamentales.

Desafíos de SOAP y WSDL

- **Verbocidad:** Los XML son grandes y pesados.
- **Complejidad:** Curva de aprendizaje elevada.
- **Parsing:** Alto consumo de CPU para procesar XML.
- **Dureza:** Un cambio mínimo en el WSDL rompe al cliente.

Computación en la Nube (Cloud Computing)

Definición de Cloud Computing

- **Definición NIST:** Modelo para habilitar acceso ubicuo, conveniente y bajo demanda a un fondo compartido de recursos computacionales configurables.
- Los recursos (redes, servidores, almacenamiento) se pueden aprovisionar rápidamente.
- Requiere un esfuerzo mínimo de gestión o interacción con el proveedor.
- Es la culminación de la virtualización y los sistemas distribuidos.
- No es solo "el ordenador de otro", es **autoservicio escalable**.

Características de la Nube

Cinco Características Esenciales (NIST)

- 1. Autoservicio bajo demanda (On-demand self-service).
- 2. Acceso amplio a la red (Broad network access).
- 3. Compartición de recursos (Resource pooling).
- 4. Elasticidad rápida (Rapid elasticity).
- 5. Servicio medido (Measured service).

1. Autoservicio Bajo Demanda

- El usuario puede proveerse de capacidades de computación de forma automática.
- No requiere interacción humana con el proveedor de servicios.
- Se gestiona a través de portales web o APIs.
- Permite una respuesta inmediata a necesidades de cómputo.

2. Acceso Amplio a la Red

- Las capacidades están disponibles en la red.
- Se accede a través de mecanismos estándar.
- Soporta plataformas heterogéneas (Thin o Thick clients).
- Móviles, tablets, laptops y estaciones de trabajo.
- Independencia de la ubicación geográfica del usuario.

3. Compartición de Recursos (Pooling)

- Los recursos sirven a múltiples consumidores usando un modelo de **Multi-alquiler** (Multi-tenancy).
- Recursos físicos y virtuales se asignan dinámicamente según la demanda.
- El cliente generalmente no sabe exactamente dónde reside el recurso (Data Center).
- Ejemplos: Almacenamiento, memoria, ancho de banda.

4. Elasticidad Rápida

- Las capacidades pueden crecer o disminuir de forma automática.
- Para el consumidor, los recursos disponibles parecen ser **infinitos**.
- Se adapta a picos de tráfico (ej. ventas de fin de año).
- Evita el sobre-aprovisionamiento y el infra-aprovisionamiento.

5. Servicio Medido (Pay-as-you-go)

- Los sistemas de nube controlan y optimizan el uso de recursos.
- Se basa en una capacidad de medición (Metering).
- Transparencia tanto para el proveedor como para el consumidor.
- Se paga solo por lo que se consume (minutos de CPU, GB de datos).

Modelos de Servicio

La Pirámide de Modelos de Servicio

- Los servicios de nube se dividen según el nivel de abstracción.
- **SaaS:** Software como Servicio (Capa Superior).
- **PaaS:** Plataforma como Servicio (Capa Media).
- **IaaS:** Infraestructura como Servicio (Capa Base).
- Cada modelo transfiere diferentes responsabilidades al proveedor.

Concepto de IaaS

- Provee recursos de computación fundamentales.
- Servidores virtuales, almacenamiento y redes.
- El consumidor tiene control sobre el SO, almacenamiento y aplicaciones instaladas.
- No gestiona la infraestructura física subyacente.
- Es el modelo más cercano al hardware.

Componentes de IaaS

- **Cómputo:** Máquinas virtuales (EC2, Azure VM).
- **Almacenamiento:** Discos virtuales y objetos (S3, Block Storage).
- **Redes:** Virtual Private Clouds (VPC), Firewalls, IPs.
- **Escalabilidad:** Capacidad de añadir más nodos manualmente.

Responsabilidades en IaaS

- **Proveedor:** Hardware, Data Center, Virtualización.
- **Usuario:** Sistema Operativo, Parches de seguridad, Middleware, Datos, Aplicaciones.
- *Ideal para: Administradores de sistemas y DevOps.*

Concepto de PaaS

- Provee un entorno de desarrollo y despliegue.
- El consumidor despliega aplicaciones creadas con lenguajes y herramientas soportadas.
- Se abstrae la gestión del Sistema Operativo y el Servidor.
- Permite enfocarse exclusivamente en el **Código**.

Beneficios de PaaS

- Reducción de la complejidad en el despliegue.
- Escalado automático de la aplicación.
- Servicios integrados: Bases de datos, colas de mensajes, autenticación.
- Ejemplos: Google App Engine, Heroku, AWS Elastic Beanstalk.

Responsabilidades en PaaS

- **Proveedor:** SO, Runtime (Java, Python, Node.js), Middleware, Virtualización.
- **Usuario:** Código de la aplicación y configuración de datos.
- *Ideal para: Desarrolladores de Software.*

Concepto de SaaS

- Aplicaciones listas para ser usadas por el usuario final.
- Se accede generalmente a través de un navegador web.
- El consumidor no gestiona ni controla la infraestructura ni la plataforma.
- Modelo de suscripción o gratuito.

Ejemplos y Casos de Uso de SaaS

- **Productividad:** Google Workspace, Microsoft 365.
- **CRM:** Salesforce.
- **Almacenamiento:** Dropbox, Google Drive.
- **Comunicación:** Slack, Zoom.
- Todo el mantenimiento y las actualizaciones son invisibles para el usuario.

Responsabilidades en SaaS

- **Proveedor:** Gestiona TODO el stack tecnológico.
- **Usuario:** Solo gestiona su propia información y configuraciones de usuario.
- *Ideal para: Usuarios finales y empresas que buscan soluciones listas.*

Cuadro Comparativo: On-Premise vs Cloud

- **On-Premise:** Usted gestiona todo (desde el cableado hasta el software).
- **IaaS:** Usted gestiona el SO y el software.
- **PaaS:** Usted gestiona solo el software.
- **SaaS:** Usted no gestiona nada técnico.

Modelos de Despliegue (Breve)

- **Nube Pública:** Recursos abiertos al público general (AWS, Azure).
- **Nube Privada:** Uso exclusivo para una organización.
- **Nube Híbrida:** Combinación de ambas con portabilidad de datos.
- **Multicloud:** Uso de múltiples proveedores para evitar el "Vendor Lock-in".